arXiv:2501.15614v1 [quant-ph] 26 Jan 2025

Exponential Improvement on Asian Option Pricing Through Quantum Preconditioning Methods

Gumaro Rendon, Rutuja Kshirsagar, Quoc Hoan Tran

Abstract

In this work, we present a quantum algorithm designed to solve the differential equation used in the pricing of Asian options, in the framework of the Black-Scholes model. Our approach modifies an existing quantum pre-conditioning method (different from classical methods) for the problem of Asian option pricing such that we remove the dependence on the original condition number of discretized differential equation (system of linear equations). This was possible with new fast-forwardable discretizations of the first and second derivatives with respect to the underlying asset value ratio (value over average). We determine that these discretizations handle well kinks in the initial/terminal conditions. We also introduce a new circuit construction for the discretized time-derivative operator with Dirichlet boundary conditions which avoids the oracle workspace needed for the general sparse matrix implementation. Here, we also devised a new method probability integral estimation from which we extract the solution, achieving $\tilde{O}(\text{polylog}\,(1/\epsilon))$, which is an exponential improvement over other quantum methods when it comes to solution information extraction from the solution state.

1 Introduction

As quantum computing continues to gain momentum, researchers are exploring its potential across a variety of fields, with finance emerging as one of the most promising areas for transformative applications. Quantum algorithms are seen as having the ability to revolutionize traditional financial practices [19, 29, 32, 34, 43, 44, 3, 10, 1, 5, 45], offering substantial improvements over classical methods in several critical tasks, such as option pricing. Option pricing is essential for predicting the value of financial assets, helping investors make more informed decisions. Given the complexity of financial markets, especially in the realm of derivatives, quantum computing presents an opportunity to significantly enhance the accuracy and efficiency of financial modeling.

Option pricing is a fundamental problem in finance, involving the determination of the fair value of contracts about the right to buy or sell an asset at an agreed price, which depend on the price movements of underlying assets over time. The Black-Scholes model, a widely recognized framework, uses a partial differential equation to describe how an option's price evolves, accounting for factors like the asset price, volatility, time to contract expiration, and interest rates. However, this model assumes constant volatility, which may not reflect real market dynamics, prompting the development of more sophisticated models such as the Heston model, which incorporates stochastic volatility. To solve these equations, methods ranging from classical analytical solutions to numerical techniques like Monte Carlo simulations are employed, with the latter being particularly useful but computationally expensive. This has spurred interest in quantum computing, with quantum algorithms potentially offering a "quantum advantage" in speeding up Monte Carlo simulations. In the case of Asian options, which derive their payoff from the average price of an underlying asset over a period rather than its price at expiration, the Black-Scholes model can be adapted, though it requires adjustments to account for the correlation between the asset's price at different points in time. Valuing Asian options under the Black-Scholes framework typically involves more complex analytical or numerical methods, such as Monte Carlo simulations, to handle the averaging effect.

This work focuses specifically on the use of quantum algorithms to price Asian options. Unlike standard vanilla options, where the payoff is determined at a single point in time, Asian options are a type of average option where the final payoff depends on the average price of the underlying asset over a specified time

period. These options tend to be less volatile, making them an attractive choice for more stable investment strategies, particularly within the commodities derivatives market.

The primary goal of this research is to develop and implement a quantum algorithm for pricing Asian options under the Black-Scholes model, a cornerstone in financial modeling. While a number of classical [48, 38, 14, 20, 39] and quantum [31, 16, 49, 47] algorithms have already been proposed for this task, the implementation of quantum approaches in option pricing is still an area of active research. Our approach builds on and expands existing quantum methods, aiming to offer an efficient and scalable solution for pricing Asian options. The key steps in our algorithm are summarized in Figure 2 and the key improvements in our work are shown in Figure 1.

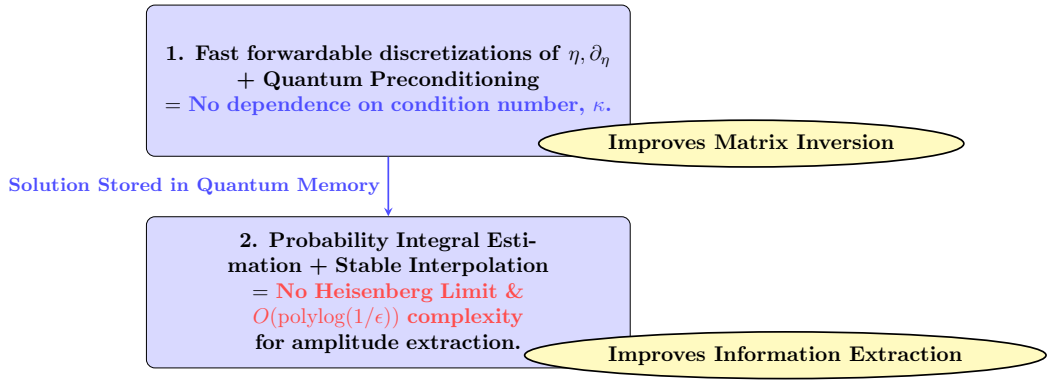

Figure 1: Addressing bottlenecks in solving the Black-Scholes equation for Asian Option Pricing.

In a foundational paper by Rebentrost et al. [33], the authors introduced a quantum Monte Carlo simulation algorithm tailored for the Black-Scholes-Merton option pricing model, which applies to both plain vanilla and Asian options. This model assumes that the underlying asset follows a martingale distribution, a common assumption in financial modeling. The quantum algorithm proposed by Rebentrost and colleagues achieves a quadratic speedup in the number of samples required, outperforming classical Monte Carlo methods that are typically used to simulate the underlying asset's price dynamics.

Another notable contribution is from Wolf et al. [49], who proposed a quantum algorithm for pricing discrete floating-strike Asian options using a valuation tree. This approach minimizes circuit complexity by reducing the number of qubits required at each time step. By avoiding the need to discretize identical bit values at different intervals, the algorithm achieves a quadratic speedup compared to classical Monte Carlo simulations. Sajid et al. [2] further demonstrated the algorithm's implementation using the Qiskit simulator, validating its potential for practical use in quantum finance.

In a more recent work by Prakash et al. [30], the authors proposed a quantum algorithm for pricing Asian options under a discrete monitoring scheme. Their algorithm operates with a poly-logarithmic complexity in the number of discrete monitoring points, along with polynomial complexity in the inverse of the additive error, making it both efficient and scalable for large datasets. This work demonstrates the advantages of quantum computing in handling complex, multi-dimensional option pricing problems.

Further extending the potential of quantum algorithms in finance, Wang et al. [47] introduced quantum algorithms for pricing Asian and barrier options under the stochastic volatility (Heston) model. Their approach leverages the flexibility of quantum computing to model more complex market dynamics, offering a quadratic speedup over classical methods. The authors also computed the gate complexities for the Clifford+T basis, with detailed results provided in Table 6 of their study.

To better understand the quantum advantages in this domain, we have compiled an expanded [30] list of key quantum and quantum-inspired algorithms for solving differential equations in finance. We have also included the best known classical algorithm tailored for Asian Option Pricing at a complexity scaling of $O(1/\epsilon)$ [9, 25]. This compilation highlights their respective computational complexities and qubit requirements, which are summarized in Table 1.

In this work, we present a quantum algorithm that takes a different approach. Instead of finding speed-ups on solving the Stochastic Differential Equation through quantum Monte Carlo (quadratic speed-up through

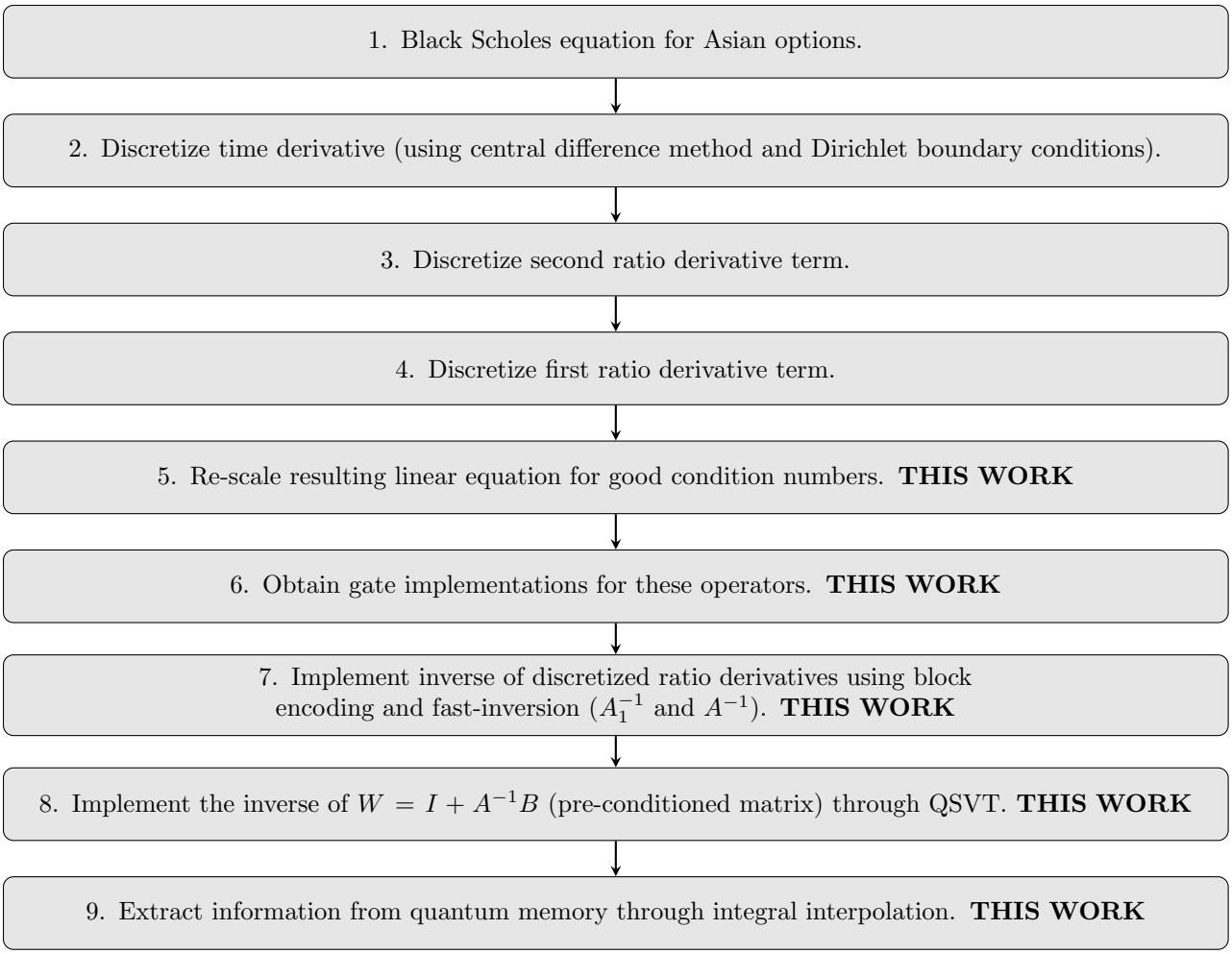

Figure 2: The above flowchart represents various steps in obtaining our algorithm.

Grover-like algorithms) for solving the corresponding Stochastic Differential Equation, we use quantum pre-conditioning methods and a new amplitude extraction to find the solution of the corresponding Ordinary Differential Equation. One can obtain the ODE from the corresponding SDE using the Feynman-Kac/Fokker-Planck formula for parabolic equations, in a more general sense, one can use the Kramer-Moyal expansion of the master equation (sometimes called forward/backward-Kolmogorov equations) for probability density evolution of a Markov-chain process.

Among recent broader quantum algorithms results, [11], authors compared different classical and quantum methods for solving linear differential equations. For the classical best methods they list regular FEM (Finite Element Method), FVM (Finite Volume Method), and FDM (Finite Difference Method), their adaptive counterparts [4], and some sparse grid methods, spectral an otherwise. The adaptive FDM/FEM and spectral sparse grid [41, 40] methods reaching $O(\text{polylog}(1/\epsilon))$; these complexity scalings are meant for smooth solutions. In the presence of singularities, when considering the scaling of the condition number with ϵ, the complexity scaling for these classical methods degrades to $O\left(\text{poly}(1/\epsilon)\right)$, even in adaptive meshes. The same happens to the quantum spectral and adaptive FDM methods proposed in this work [11], as they assume the solution has finite higher-order derivatives.

Moreover, the authors of [11] only consider the quantum complexity cost of preparing the quantum state ϵ-close to the exact solution on a mesh. This is an assumption that is made since the seminal work of [18] because, as correctly pointed out by [28], the general extraction of the solution through amplitude estimation [6] is bounded by the Heisenberg limit. Here, we describe a new method of extracting the integral

Algorithm	Computation Cost	Qubit Requirements
This work	$\tilde{O}(\text{polylog}\,(1/\varepsilon))$	$O\,(\text{polylog}\,(1/\varepsilon))$
Best classical methods for AOP [9, 25]	$O(1/\varepsilon)$	NA
Valuation tree method [49]	$O(1/\varepsilon^2)$	$O(1/\varepsilon)$
Semi-digital encoding [30]	$\tilde{O}(1/\varepsilon^4)$	$\tilde{O}(1/\epsilon^2)$
Quantum-inspired sampling [30]	$\tilde{O}(1/\varepsilon^6)$	NA
Time-domain sub-sampling [30]	$\tilde{O}(1/\varepsilon^4)$	NA
Quantized sub-sampling [30]	$\tilde{O}(1/\varepsilon^3)$	$\tilde{O}(1/\varepsilon^2)$
Standard quantum Monte-Carlo Integration [33, 30]	$\tilde{O}(1/\varepsilon^2)$	$\tilde{O}(1/\varepsilon)$
Standard classical Monte-Carlo Integration [30]	$\tilde{O}(1/\varepsilon^3)$	NA

Table 1: Complexity of various algorithms for Asian option pricing; ε is an additive error. For Monte Carlo approaches (solving the corresponding stochastic differential equation) we have assumed first-order weak convergence rates like the Euler-Maruyama or Milstein time integration methods.

probabilities for positive definite solutions, smooth on the domain considered, that avoids this limit.

The quantum computational complexities displayed here are only in terms of ϵ, the error on the extracted solution, since that is ultimately what one cares about. The scaling with respect to the condition number for the obtained linear system is not displayed because its dependence has been eliminated through quantum pre-conditioning [46], a new quantum method for a particular kind of linear systems. Similarly, we cast computational complexities from other work with which we compare in terms of ϵ only.

In the context of quantum algorithms, the use of Chebyshev interpolation has gained some relatively new importance in the context of hybrid methods for Hamiltonian evolution [37, 35] and in option pricing [26]. The stability and the convergence rates make it an attractive choice for interpolating smooth functions. For option pricing (see [26]), with these interpolation methods they achieve a quadratic advantage over classical Monte Carlo methods for Bermudan option pricing. This was an improvement over the first theorization of quantum speed-up of Monte Carlo methods [27]. Here, Chebyshev interpolation, instead allowing us to approach the Heisenberg limit, will provide an exponential advantage by interpolating the integrated probabilities (See Section 4.1).

Overall, quantum computing offers a promising future for financial applications, particularly in option pricing, by providing faster, more efficient solutions to problems that are computationally expensive for classical methods. As these quantum algorithms continue to evolve, they have the potential to fundamentally reshape the landscape of financial modeling and risk management.

The rest of the paper is organized as follows. Section 2 contains the preliminaries to be used throughout the paper. Section 3 contains major parts of our algorithm which consist of explaining our specific discretization strategy for the operators in the differential equation for Asian options pricing, as well as the use of fast-inversion for fast-fowardable discretized operators which help us pre-condition the linear system at hand. Section 4 is the amplitude extraction part of the algorithm which is comprised of a new probability integral method through binary decomposition and the use of slightly-off Chebyshev interpolation. Section 5 provides a summary of the methodology developed in the paper. Section 6 is the conclusion and future direction section.

2 Preliminaries

In this section, we summarize few definitions and results that are important for developing our method. We introduce more notations in Sections 3 and 4 as and when necessary.

2.1 Asian Options Pricing under Black-Scholes model

This work focuses on the mathematical model used to price Asian options, which are a type of financial derivative where the payoff depends on the average price of the underlying asset over a period of time, rather than just its price at expiration. The model is based on a modified version of the Black-Scholes equation,

which is commonly used for standard options but needs adjustments to account for the averaging effect in Asian options. The key idea is to calculate the option's value by solving this equation, where the parameters include factors like volatility, risk-free rate, and dividend yield, along with the average price of the asset over time.

We look at the Black-Scholes differential Equation (1) as defined in [15],

$$\frac{\partial \psi}{\partial \tau_1} - \frac{1}{2}\sigma^2\eta^2\frac{\partial^2 \psi}{\partial \eta^2} - \left(\frac{W(\tau_1)}{T} - (r-q)\eta\right)\frac{\partial \psi}{\partial \eta} = 0, \tag{1}$$

where ψ is the adjusted payoff, η is the ratio of the average versus underlying (that is I/ST) for average-strike options or the ration of the average minus the strike (i.e. $(I-K)/ST$) for the average-rate options, S is the price of the underlying asset, K is the strike price for average-rate Asian options, $\tau_1 = T - t$ time variable after applying time reversal, $W(\tau_1)$ is a weight function for the average $I(\tau_1) = \int_0^{\tau_1} W(\tau_1')S(\tau_1')\mathrm{d}\tau_1'$, σ is the volatility, r is the constant risk-free rate, q is the constant dividend yield, and T is the expiry time.

To determine the option price, time reversal is applied, which helps translate the problem into an initial value problem. The specific payoffs for different types of Asian options (average-rate and average-strike) are defined at the start, and these payoffs drive the solution of the equation. Once the equation is solved, the option's value can be calculated using a formula that incorporates the average price of the underlying asset, adjusting for time and dividends. This approach allows for accurate pricing of Asian options, which are more complex than traditional options due to their reliance on average prices.

After the time reversal the terminal conditions become the following initial conditions for the four different cases:

1. average-rate call: $\psi(\eta, 0) = \max(\eta, 0)$;
2. average-rate put: $\psi(\eta, 0) = \max(-\eta, 0)$;
3. average-strike call: $\psi(\eta, 0) = \max(1-\eta, 0)$;
4. average-strike put: $\psi(\eta, 0) = \max(\eta - 1, 0)$;

Finally, we recover the option price through

$$V(S, I, t) = Se^{-q(T-t)}\psi\left(\frac{I - KT}{ST}, T - t\right) \tag{2}$$

for average-rate options and

$$V(S, I, t) = Se^{-q(T-t)}\psi\left(\frac{I}{ST}, T - t\right) \tag{3}$$

for average-strike options.

2.2 Pre-conditioning

In this subsection, we focus on quantum pre-conditioning for solving linear systems, as discussed in [46]. The central idea is to improve the condition number of a linear system by reformulating it into an equivalent system with a smaller condition number, making it easier to solve. Consider the linear system in Equation (4) for matrices A, B and states $|x\rangle, |b\rangle$,

$$(A + B)|x\rangle = |b\rangle. \tag{4}$$

In this system, we assume that $\|A\|$ can be very large, and that $\|B\|$, $\|(A+B)^{-1}\|$, and $\|A^{-1}\|$ are $O(1)$.

Now, with these assumptions, it is possible to reduce the condition number of the linear system by considering the following equivalent formulation

$$(I + A^{-1}B)|x\rangle \propto A^{-1}|b\rangle. \tag{5}$$

A key result from [46] is restated in Lemma 1, explains why linear system in Equation (5) might have a much smaller condition number than the linear system in Equation (4). Lemma 1 provides an upper bound for the condition number of the pre-conditioned system. It states that the condition number of the matrix $W = I + A^{-1}B$, which represents the pre-conditioned system, is bounded by products of certain norms involving A and B. Specifically, the condition number $\kappa(W)$ depends on the norms of A^{-1} and B, and is independent of $||A||$, which is typically the source of ill-conditioning in the original system.

Lemma 1. *Define $W = I + A^{-1}B$, then the smallest singular value $\sigma_{\min}$ and largest singular value $\sigma_{\max}$ of W satisfy*

$$1/\sigma_{\min} \leq 1 + \|(A+B)^{-1}\|\|B\| =: C_{AB}$$
$$\sigma_{\max} \leq 1 + \|A^{-1}\|\|B\| =: C'_{AB}$$

Hence the condition number of W, $\kappa(W)$ can be upper bounded as

$$\kappa(W) \leq \left(1 + \|(A+B)^{-1}\|\|B\|\right)\left(1 + \|A^{-1}\|\|B\|\right) = C_{AB}C'_{AB}. \tag{6}$$

We also layout their proof for the sake of the reader:

Proof. Let $W|x\rangle = |y\rangle$. Then we have

$$(A+B)|x\rangle = A|y\rangle,$$

therefore

$$(A+B)(|x\rangle - |y\rangle) = -B|y\rangle$$
$$A(|x\rangle - |y\rangle) = -B|x\rangle,$$

and these two equalities lead to

$$\| |x\rangle \| \leq \| |y\rangle \| + \| |x\rangle - |y\rangle \| \leq (1 + \|(A+B)^{-1}\|\|B\|)\| |y\rangle \|$$
$$\| |y\rangle \| \leq \| |x\rangle \| + \| |x\rangle - |y\rangle \| \leq (1 + \|A^{-1}\|\|B\|)\| |x\rangle \|.$$

□

3 Discretization of the Differential Equation and Fast Inversion

In this section we explain our choice of discretization for the differential equation in Equation (1). We explain a new circuit implementation for our choice of discretization for the time derivative with Dirichlet boundary conditions. Here is also a detailed explanation of the fast-inversion [46] algorithm in the presence of a fast-forwardable evolution for a hermitian matrix. Finally, in this section we also summarize the qubit requirements for all the block-encodings involved in our implementation.

3.1 Discretization of Asian Options Equation

In order to solve the differential equation, we approach the problem through the finite difference method. That is, we approximate the derivatives in Equation (1) with finite differences. How we discretize these operators will largely determine the circuit implementations, their gate cost, and the possibility of us using methods like the one detailed in Section 2.2. In this subsection, we look at the discretization of the various terms in Equation (1). We start with discretizing the first term in Equation (1), the time derivative. For the time derivative, we use central difference with Dirichlet boundary conditions. This discretized derivative operator, on time coordinates only, looks the following

$$\tilde{C}_{\tau_1} = \frac{1}{2\delta_{\tau_1}} \begin{pmatrix} 0 & 1 & \dots & 0 & 0 \\ -1 & 0 & 1\dots & 0 & 0 \\ 0 & -1 & 0\dots & 0 & 0 \\ \vdots & \vdots & \vdots & & \\ 0 & 0 & \dots & 0 & 1 \\ 0 & 0 & \dots & -1 & 0 \end{pmatrix} \tag{7}$$

Here, $\delta_{\tau_1} = 1/(2^{n_{\tau_1}} + 1)$ are the number of steps of the time interval $[0, T]$ and n_{τ_1} are the number of qubits required to encode it. Let I_η be the identity matrix acting on the η-space of size 2^{n_η}. Here, 2^{n_η} is the number of discretization points in the η variable, and n_η is the number of qubits representing it. This way, the Equation (1) will look of the form

$$\left(\tilde{C}_{\tau_1} \otimes I_\eta + \dots\right) |\psi\rangle = |\tilde{b}\rangle \,. \tag{8}$$

Here the elements of $|\psi\rangle$ approximate the solution ψ without any re-scaling, that is, $\langle\psi|\eta, \tau_1\rangle \approx \psi(\eta, \tau_1)$. Moreover, $|\tilde{b}\rangle$ encodes the initial and terminal conditions stemming from the initial/final finite difference terms not included in $\tilde{C}_{\tau_1}$. Instead, we have moved it to the right hand-side of the equation. That is, the initial and final conditions in Equation (1) will be implemented through

$$|\tilde{b}\rangle = \frac{1}{2\delta_{\tau_1}} \begin{pmatrix} \psi_0 \\ 0 \\ 0 \\ \vdots \\ 0 \\ -\psi_0 \end{pmatrix}. \tag{9}$$

We have chosen to use the same boundary condition for the initial and final conditions $\psi(\eta, 0) = \psi(\eta, T) = \psi_0$, where $\psi(\eta, 0)$ is determined by the problem at hand as stated in Section 2.1.

Following this, we now discretize the second term in Equation (1). For that, we fix the nodes on the discretized η variable with interval spacings of size

$$\delta_\eta = \frac{2\eta_{\max}}{2^{n_\eta}}. \tag{10}$$

Here, $\eta_{\max}$ are the bounds on the variable η at which we truncate the system. These nodes are collocated at $\eta \in \left\{-\left(\eta_{\max} - \frac{\delta_\eta}{2}\right), \dots, -3\frac{\delta_\eta}{2}, -\frac{\delta_\eta}{2}, \frac{\delta_\eta}{2}, 3\frac{\delta_\eta}{2}, \dots, \left(\eta_{\max} - \frac{\delta_\eta}{2}\right)\right\}$. Now, we define a normalized $\eta/\eta_{\max}$ operator:

$$\hat{\eta} = \left(\hat{\delta}_\eta/2\right) I_\eta + \begin{pmatrix} -1 & 0 & 0 & \dots & 0 \\ 0 & -1+\delta_\eta & 0 & \dots & 0 \\ 0 & 0 & -1+2\delta_\eta & \dots & 0 \\ \vdots & \vdots & \dots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & 1-\delta_\eta \end{pmatrix}, \tag{11}$$

where $\hat{\delta}_\eta = 2/2^{n_\eta}$. This matrix, we can express as a linear combination Pauli operators through

$$\hat{\eta} = \left(\hat{\delta}_\eta/2 - 1\right) I_\eta + \frac{2}{2^{n_\eta}} \sum_{j=0}^{n_\eta - 1} 2^{j+1} Z_j, \tag{12}$$

where Z_j are the Pauli Z-gate,

$$\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \tag{13}$$

acting on the j-th qubit. We also used the following discretization for the ∂_η operator based on the Fourier transform:

$$\Delta_\eta = -i \left(\frac{\pi}{\hat{\delta}_\eta \eta_{\max}}\right) F_c^\dagger \hat{\eta} F_c, \tag{14}$$

which we obtain in Appendix B. Here, $\pm\eta_{\max}$ are the values of η at which we truncate its domain. Here, F_c is the centered discretized Fourier transform, representing a $2^{n_\eta} \times 2^{n_\eta}$ matrix. These are centered discretizations of the Fourier transform similar to what is used in [42, 21, 36]. Explicitly, in ket notation:

$$F_c = \frac{1}{\sqrt{2^{n_\eta}}} \sum_{k=-(2^{n_\eta}-1)/2}^{(2^{n_\eta}-1)/2} \sum_{j=-(2^{n_\eta}-1)/2}^{(2^{n_\eta}-1)/2} \left(e^{-i2\pi/2^{n_\eta}}\right)^{kj} |k\rangle \langle j| \,. \tag{15}$$

where j and k are running over half indices $j,k \in \{-\frac{2^{n_\eta}-1}{2},\dots,-\frac{\hat{\delta}_\eta}{2},\frac{\hat{\delta}_\eta}{2},\dots,\frac{2^{n_\eta}-1}{2}\}$. This discretization correctly handle kinks (derivative discontinuities) in $\psi_0(\eta)$, provided that the kink in $\psi_0(\eta)$ is off the discretization nodes (See Appendix B). With this, we can discretize the second term in Equation (1) (the one with the second derivative of ψ with respect to η) through:

$$\begin{aligned}\tilde{C}_{\eta,1} = &\frac{\pi^2\sigma^2}{2\hat{\delta}_\eta^2}\hat{\eta}^2 \\ &\times \tilde{F}_c^\dagger \hat{\eta}^2 \tilde{F}_c.\end{aligned} \tag{16}$$

The discretization of the third term in Equation (1) is

$$\begin{aligned}&\tilde{C}_{\eta,2} = \\ &-i\frac{\pi(r-q)}{\hat{\delta}_\eta}\left(\hat{\eta}+\left(-\frac{1}{\eta_{\max}T(r-q)}\right)I_\eta\right) \\ &\times \tilde{F}_c^\dagger(\hat{\eta})\tilde{F}_c.\end{aligned} \tag{17}$$

With this, our linear equation reads:

$$\left(\tilde{C}_{\tau_1}\otimes I_\eta + I_{\tau_1}\otimes \tilde{C}_{\eta,1} + I_{\tau_1}\otimes \tilde{C}_{\eta,2}\right)|\psi\rangle = |\tilde{b}\rangle\,, \tag{18}$$

here I_{τ_1} is the identity matrix of size $2^{n_{\tau_1}}\times 2^{n_{\tau_1}}$. Thus, the problem can be stored in a $2^{n_{\tau_1}+n_\eta}$ vector space or in $n_{\tau_1}+n_\eta$ qubits.

3.2 Implementing C_{τ_1}

In this subsection, we will show how to synthethize the discrete time-derivative (τ_1 is our time variable) operator (Equation (7)) needed for the discretized differential equation Equation (18).

Let $N_{\tau_1} = 2^{n_{\tau_1}}$ be the number of discretization points on the τ_1 variable. We first look at implementing the following operator for a discretized derivative with periodic boundary conditions

$$\tilde{C}_{\tau_1,\mathrm{PBC}} f(x) = \frac{f(x+\delta_{\tau_1,\mathrm{PBC}}) - f(x-\delta_{\tau_1,\mathrm{PBC}})}{2\delta_{\tau_1,\mathrm{PBC}}} \tag{19}$$

where $f(x+N_{\tau_1}\delta_{\tau_1,\mathrm{PBC}}) = f(x)$. Such discrete operator can be written in the following matrix form:

$$\tilde{C}_{\tau_1,\mathrm{PBC}} = \frac{1}{2\delta_{\tau_1,\mathrm{PBC}}}\begin{pmatrix} 0 & 1 & 0 & 0 & \dots & 0 & -1 \\ -1 & 0 & 1 & 0 & \dots & 0 & 0 \\ 0 & -1 & 0 & 1 & \dots & 0 & 0 \\ 0 & 0 & -1 & 0 & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \dots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \dots & 0 & 1 \\ 1 & 0 & 0 & 0 & \dots & -1 & 0 \end{pmatrix}, \tag{20}$$

where

$$\delta_{\tau_1,\mathrm{PBC}} = \frac{1}{N_{\tau_1}} = \frac{1}{2^{n_{\tau_1}}}. \tag{21}$$

To describe it in terms of quantum gates, we can start by splitting like this:

$$\tilde{C}_{\tau_1,\mathrm{PBC}} = \frac{1}{2\delta_{\tau_1,\mathrm{PBC}}}\begin{pmatrix} 0 & 1 & 0 & 0 & \dots & 0 & 0 \\ -1 & 0 & 0 & 0 & \dots & 0 & 0 \\ 0 & 0 & 0 & 1 & \dots & 0 & 0 \\ 0 & 0 & -1 & 0 & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \dots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \dots & 0 & 1 \\ 0 & 0 & 0 & 0 & \dots & -1 & 0 \end{pmatrix} + \frac{1}{2\delta_{\tau_1,\mathrm{PBC}}}\begin{pmatrix} 0 & 0 & 0 & 0 & \dots & 0 & -1 \\ 0 & 0 & 1 & 0 & \dots & 0 & 0 \\ 0 & -1 & 0 & 0 & \dots & 0 & 0 \\ 0 & 0 & 0 & 0 & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \dots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \dots & 0 & 0 \\ 1 & 0 & 0 & 0 & \dots & 0 & 0 \end{pmatrix}. \tag{22}$$

The first term in Equation (22) can be represented through

$$\frac{i}{2\delta_{\tau_1,\mathrm{PBC}}} I_{2^{n_{\tau_1}-1}} \otimes Y, \tag{23}$$

where Y is the gate corresponding to the σ_y-Pauli matrix acting on the least-significant qubit, $I_{2^{n_{\tau_1}-1}}$ is the identity matrix acting on the $n_{\tau_1} - 1$ most significant qubits of the τ_1-register. For the second term in Equation (22), we note that we can cyclically shift both indices by one to the right and obtain exactly the same expression. Thus, the second term can be implemented through

$$\frac{i}{2\delta_{\tau_1,\mathrm{PBC}}} S_{+1} \left(I_{2^{n_{\tau_1}-1}} \otimes Y \right) S_{+1}^{\dagger}, \tag{24}$$

where S_{+1} is the cyclical shift operator to the right. The problem with this operator is that this operator implements periodic boundary conditions, Equation (19), but we require Dirichlet (fixed) boundary conditions, $\psi(\eta, 0) = \psi_0$. That is, the desired operator is in fact

$$\tilde{C}_{\tau_1} = \frac{1}{2\delta_{\tau_1}} \begin{pmatrix} 0 & 1 & 0 & 0 & \dots & 0 & 0 \\ -1 & 0 & 1 & 0 & \dots & 0 & 0 \\ 0 & -1 & 0 & 1 & \dots & 0 & 0 \\ 0 & 0 & -1 & 0 & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \dots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \dots & 0 & 1 \\ 0 & 0 & 0 & 0 & \dots & -1 & 0 \end{pmatrix}, \tag{25}$$

and, because we now have Dirichlet boundary conditions, the spacing in between the discretization nodes in the variable τ_1 is

$$\delta_{\tau_1} = 1/(N_{\tau_1} + 1) = 1/(2^{n_{\tau_1}} + 1). \tag{26}$$

We now do not have the non-zero corner elements. We can first implement the periodic version with the two gate implementation above, except $\delta_{\tau_1,\mathrm{PBC}} \to \delta_{\tau_1}$. To cancel the corner terms, we first look at the next two matrices:

$$\frac{1}{2\delta_{\tau_1}} \begin{pmatrix} 1 & 0 & 0 & 0 & \dots & 0 & 0 \\ 0 & 1 & 0 & 0 & \dots & 0 & 0 \\ 0 & 0 & 1 & 0 & \dots & 0 & 0 \\ 0 & 0 & 0 & 1 & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \dots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \dots & 0 & -i \\ 0 & 0 & 0 & 0 & \dots & i & 0 \end{pmatrix}, \quad \frac{1}{2\delta_{\tau_1}} \begin{pmatrix} 1 & 0 & 0 & 0 & \dots & 0 & 0 \\ 0 & 1 & 0 & 0 & \dots & 0 & 0 \\ 0 & 0 & 1 & 0 & \dots & 0 & 0 \\ 0 & 0 & 0 & 1 & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \dots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \dots & 0 & i \\ 0 & 0 & 0 & 0 & \dots & -i & 0 \end{pmatrix}. \tag{27}$$

These two matrices, distinct even under global phase change, can be implemented through $C^{n_{\tau_1}-1}Y$ and $C^{n_{\tau_1}-1}(-Y)$, respectively. Here, $C^{n_{\tau_1}-1}U$ means the unitary is controlled on the $n-1$ most significant qubits. Thus, we can implement C_{τ_1} through the four terms:

$$\tilde{C}_{\tau_1} = \frac{i}{2\delta_{\tau_1}} I_{2^{n_{\tau_1}-1}} \otimes Y + \frac{i}{2\delta_{\tau_1}} S_{+1} \left(I_{2^{n_{\tau_1}-1}} \otimes Y + \frac{1}{2} C^{n_{\tau_1}-1} Y - \frac{1}{2} C^{n_{\tau_1}-1}(-Y) \right) S_{+1}^{\dagger}. \tag{28}$$

3.3 Fast Inversion

In order to implement the preconditioning of the linear system like discussed in Section 2.2, we first need to identify the matrix A in the system Equation (38) which must also be fastly invertible in order to not introduce prohibitive costs. Here, we assume that $\|A\|$ is very large and that $\|A^{-1}\| = O(1)$, we will also assume that A is fast-forwardable, that is, the cost of simulating e^{itA} is independent of the norm of A and the size of t.

Largely following the procedure from [18], we start by decomposing $|b\rangle$ in the eigenvector basis, using phase estimation [24, 13, 7]. Denote by $|u_j\rangle$ the eigenvectors of A (or equivalently, of e^{iAt}), and by λ_j the corresponding eigenvalues. Let

$$|\Psi_0\rangle := \sqrt{\frac{2}{T}} \sum_{\tau=0}^{T-1} \sin \frac{\pi(\tau + \frac{1}{2})}{T} |\tau\rangle \tag{29}$$

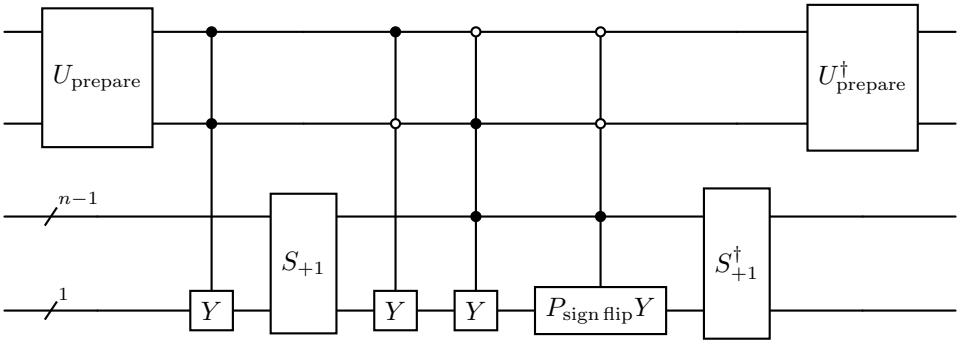

Figure 3: Circuit implementing the block-encoding of C_{τ_1}.

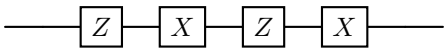

Figure 4: Circuit to implement $P_{\text{sign flip}}$. Its effects alone are unobservable because it entails a phase change overall the state, however, the state-controlled $P_{\text{sign flip}}$ is a valid observable operation.

for some large T (not to be confused with T, the expiration time used in other sections). The coefficients of $|\Psi_0\rangle$ are chosen (following [24, 7, 36]) to minimize a certain quadratic loss function which appears in the error analysis of [18]. For an efficient circuit implementation, with a cost of $O(\log(T))$, see [36].

Next we apply the conditional Hamiltonian evolution $\sum_{\tau=0}^{T-1} |\tau\rangle\langle\tau|^C \otimes e^{i\hat{A}\tau t_0/T}$ on $|\Psi_0\rangle^C \otimes |b\rangle$, where $t_0 = O(\kappa(\hat{A})/\hat{\epsilon})$, where $\hat{A} = A/\|A\|$. Fourier transforming the first register gives the state

$$\sum_{j=1}^{N}\sum_{k=0}^{T-1} \alpha_{k|j}\beta_j |k\rangle |u_j\rangle , \tag{30}$$

where $|k\rangle$ are the Fourier basis states, and $|\alpha_{k|j}|$ is large if and only if $\hat{\lambda}_j \approx \frac{2\pi k}{t_0}$. Defining $\hat{\lambda}'_k := 2\pi k/t_0$, we can relabel our $|k\rangle$ register to obtain

$$\sum_{j=1}^{N}\sum_{k=0}^{T-1} \alpha_{k|j}\beta_j |\hat{\lambda}'_k\rangle |u_j\rangle$$

Adding an ancilla qubit and rotating conditioned on $|\hat{\lambda}'_k\rangle$ yields

$$\sum_{j=1}^{N}\sum_{k=0}^{T-1} \alpha_{k|j}\beta_j |\hat{\lambda}'_k\rangle |u_j\rangle \left(\sqrt{1-\frac{C^2}{\tilde{\lambda}_k^2}}\,|0\rangle + \frac{C}{\tilde{\lambda}_k}|1\rangle\right),$$

where we choose $C = \Theta(1/\kappa(\hat{A})) < 1$. We now undo the phase estimation to uncompute the $|\hat{\lambda}'_k\rangle$. If the phase estimation were perfect, we would have $\alpha_{k|j} = 1$ if $\hat{\lambda}'_k = \hat{\lambda}_j$, and 0 otherwise. Assuming this for now, we obtain

$$\sum_{j=1}^{N} \beta_j |u_j\rangle \left(\sqrt{1-\frac{C^2}{\hat{\lambda}_j^2}}\,|0\rangle + \frac{C}{\hat{\lambda}_j}|1\rangle\right). \tag{31}$$

To finish the inversion we measure the last qubit. Conditioned on seeing 1, we have the state

$$\sqrt{\frac{1}{\sum_{j=1}^{N} C^2|\beta_j|^2/|\hat{\lambda}_j|^2}}\sum_{j=1}^{N}\beta_j\frac{C}{\hat{\lambda}_j}|u_j\rangle$$

However, if we want to actually implement the inverse of A^{-1} we must re-scale through:

$$\begin{aligned} \hat{\lambda}_j &= \lambda_j / \|A\|, \\ c = C\|A\| &= O\left(\sigma_{\min}(A)\right), \\ \epsilon &= \hat{\epsilon}/\|A\|. \end{aligned} \tag{32}$$

Thus, we have

$$\sqrt{\frac{1}{\sum_{j=1}^{N} c^2 |\beta_j|^2 / |\lambda_j|^2}} \sum_{j=1}^{N} \beta_j \frac{c}{\lambda_j} |u_j\rangle$$

which corresponds to $|x\rangle = \sum_{j=1}^{n} \beta_j \lambda_j^{-1} |u_j\rangle$ up to normalization.

We can determine the normalization factor from the probability of obtaining 1 and then taking the square root.

The error on $\hat{\lambda}_j$ coming from the phase estimation is $O(1/t_0)$. The error in $\hat{\lambda}_j^{-1}$ is $O(1/(\hat{\lambda}_j t_0))$, thus choosing $t_0 = O(\kappa(\hat{A})/\hat{\epsilon})$ ensures an error $\hat{\epsilon}$ on $\hat{\lambda}_j^{-1}$ and consequently an error ϵ on λ_j^{-1}.

To avoid aliasing, we must choose a

$$T = O(t_0) \tag{33}$$

and for an error rate p_{fail}

$$T = O(1/\sqrt{p_{\text{fail}}}). \tag{34}$$

We note that $t_0 = O\left(1/\left(\sigma_{\min}(A)\epsilon\right)\right)$ and that because of the fast-forwardability of A the cost of phase estimation and the number of ancillar qubits are both going to be $O\left(\log_2\left(\max\left(\frac{1}{\sqrt{p_{\text{fail}}}}, \frac{1}{\epsilon\sigma_{\min}(A)}\right)\right)\right)$.

With this procedure we can also block-encode A^{-1}, where the select operation is done by projecting onto measuring $|1\rangle$ on the ancillar qubit with a sub-normalization of:

$$\alpha = 1/c = \Theta\left(1/\sigma_{\min}(A)\right). \tag{35}$$

That is, the procedure by [18] lets us implement

$$U_{\text{inv}} = G + \frac{1}{\alpha} |0\rangle\langle 0| \otimes A^{-1} \tag{36}$$

with error $\epsilon_{U_{\text{inv}}} = O(\kappa(\hat{A})/t_0) = O(\epsilon)$, and for some garbage matrix G, where $\left(\langle 0| \otimes I\right) G \left(|0\rangle \otimes I\right) = 0$.

3.4 Finding A and relationships between the spacings δ_η and δ_{τ_1}

In this section we identify the matrix A of Section 2.2 in the linear equation Equation (38). The difference between Equation (38) and the assumed linear system in [46] is that here we need to do two fast inversions. The system we will have at hand is of the form:

$$A_1 \left(A + B\right) = |\hat{b}\rangle, \tag{37}$$

where A_1 and A are both fast-invertible (See Section 3.3). In order to maintain good cost-scalings throughout each inversion step, we must find the specific relationship between the scales (lattice spacings) δ_{τ_1} and δ_η that is needed to get the linear Equation (38) to the form needed for Lemma 1.

We first rescale the linear equation of by a factor of δ_{τ_1} such that Equation (18) becomes

$$\left(C_{\tau_1} \otimes I_\eta + I_{\tau_1} \otimes C_{\eta,1} + I_{\tau_1} \otimes C_{\eta,2}\right) |\psi\rangle = |b\rangle, \tag{38}$$

where

$$\begin{aligned} |b\rangle &= \delta_{\tau_1} |\tilde{b}\rangle, \\ C_{\tau_1} &= \delta_{\tau_1} \tilde{C}_{\tau_1}, \\ C_{\eta,1} &= \delta_{\tau_1} \tilde{C}_{\eta,1}, \\ C_{\eta,2} &= \delta_{\tau_1} \tilde{C}_{\eta,2}. \end{aligned} \tag{39}$$

We also note that with the initial conditions of our problem, ,we expect

$$\langle b|b\rangle = O\left(\frac{\int_{-\eta_{\max}}^{\eta_{\max}} \psi_0^2 \mathrm{d}\eta}{\delta_n}\right) = \mathcal{N}_b = O\left(\frac{\eta_{\max}^3}{\delta_n}\right). \tag{40}$$

We must normalize this vector to prepare it in a quantum computer. Moreover, we adjust the linear equation and re-absorb this new re-scaling into the vector state on the left-hand side. That is,

$$\left(C_{\tau_1} \otimes I_\eta + I_{\tau_1} \otimes C_{\eta,1} + I_{\tau_1} \otimes C_{\eta,2}\right) |\tilde{\psi}\rangle = |\hat{b}\rangle\,, \tag{41}$$

where

$$\begin{aligned} |\hat{b}\rangle &= \frac{|b\rangle}{\sqrt{\mathcal{N}_b}}, \\ |\tilde{\psi}\rangle &= \frac{|\psi\rangle}{\sqrt{\mathcal{N}_b}}. \end{aligned} \tag{42}$$

We now factorize $C_{\eta,1}$ the following way:

$$C_{\eta,1} = A_1 A_2, \tag{43}$$

where

$$\begin{aligned} A_1 &= \frac{\delta_{\tau_1} \pi^2 \sigma^2 \left(\hat{\eta}\right)^2}{2}, \\ A_2 &= \frac{1}{\hat{\delta}_\eta^2} \tilde{F}_{c,2}^\dagger \left(\hat{\eta}\right)^2 \tilde{F}_{c,2}. \end{aligned} \tag{44}$$

Before using Lemma 1 we need to apply/block-encode the inverse of A_1 on Equation (38)

$$\left(I_{\tau_1} \otimes A_2 + C_{\tau_1} \otimes A_1^{-1} + I_{\tau_1} \otimes A_1^{-1} C_{\eta,2}\right) |\tilde{\psi}\rangle = A_1^{-1} |\hat{b}\rangle\,. \tag{45}$$

which we can do through the method in Section 3.3 with a cost $\tilde{O}(\mathrm{poly}\left(\sigma_{\min}(A_1^{-1})\right)$. With this, we can identify

$$\begin{aligned} A &= I_{\tau_1} \otimes A_2 \\ B &= C_{\tau_1} \otimes A_1^{-1} + I_{\tau_1} \otimes A_1^{-1} C_{\eta,2} \end{aligned} \tag{46}$$

from Lemma 1.

To make use of the pre-conditioning approach in [46], we need

$$\|C_{\tau_1} \otimes I_\eta + I_{\tau_1} \otimes C_{\eta,2}\| = O(1), \tag{47}$$

which in turn means we need

$$\frac{\delta_{\tau_1}}{\hat{\delta}_\eta} = O(1). \tag{48}$$

Moreover, we would want $\|A_1^{-1}\| = \tilde{\Theta}(1)$, where the tilde means we allow for polylog$(\cdot)$ factors. This means we are trying to enforce

$$\frac{\hat{\delta}_\eta^2}{\delta_{\tau_1} \sigma^2} = \tilde{\Theta}(1) \tag{49}$$

as well. Thus, one solution for these constraints is that we set

$$\delta_{\tau_1} = \tilde{\Theta}\left(\frac{\hat{\delta}_\eta^2}{\sigma^2}\right). \tag{50}$$

We note also that we have factorized $C_{\eta,1}$ such that $\|A_2^{-1}\| = \Theta(1)$ regardless of the relations between δ_η, δ_{τ_1}, and $\eta_{\max}$. With this, we can always make $\|A_1^{-1}\|$ just small enough so that $\left\|(A+B)^{-1}\right\| = O(1)$ while maintaining Equation (51). We also note that we need to enforce the relation Eq. (125) in the Appendix B. Thus, finally, we get

$$\delta_{\tau_1} = \Theta\left(\frac{\hat{\delta}_\eta^2}{\sigma^2}\log 1/\epsilon\right). \tag{51}$$

3.5 Block-encodings

In this section, we detail the block-encodings needed for implementing the whole solution of Equation (38). This will help us account for all the renormalization factors, α's, and all the needed ancillar qubits (See Table 2).

First, we need the block-encoding of the fast-inversion of A_1 with the method of Section 3.3

$$U_{A_1^{-1}}\left|\hat{b}\right\rangle\left|0\right\rangle_1 = \left(\frac{A_1^{-1}}{\alpha_{A_1^{-1}}}\left|\hat{b}\right\rangle\right)\left|0\right\rangle_1 + \left(G_{A_1^{-1}}\left|\hat{b}\right\rangle\right)\left|0\right\rangle_1 \tag{52}$$

Following that, we need the unitary that block-encodes the application of A^{-1}

$$\begin{aligned} U_{A^{-1}}U_{A_1^{-1}}\left|\hat{b}\right\rangle\left|0\right\rangle_1\left|0\right\rangle_2 = &\left(\frac{A^{-1}}{\alpha_{A^{-1}}}\frac{A_1^{-1}}{\alpha_{A_1^{-1}}}\left|\hat{b}\right\rangle\right)\left|0\right\rangle_1\left|0\right\rangle_2 \\ &+\left(\frac{A^{-1}}{\alpha_{A^{-1}}}G_{A_1^{-1}}\left|\hat{b}\right\rangle\right)\left|0\right\rangle_1\left|0\right\rangle_2 + \dots \end{aligned} \tag{53}$$

Finally, we need to apply a unitary block-encoding the inverse of W where

$$W = I + A^{-1}B, \tag{54}$$

recalling that we have identified A and B in Equation (46). For that, we will need to do a signal transformation through a version of Quantum Signal Processing [17], such that we take the block-encoding of W

$$\left(\left\langle 0\right|_3 \otimes I\right) U_W \left(\left|0\right\rangle_3 \otimes I\right) = \frac{W}{\alpha_W}, \tag{55}$$

and obtain a unitary block-encoding of the inverse of W.

We now detail all the block-encodings needed for the block-encoding of W itself. We already know how to implement the the block-encoding of A^{-1}, and to implement the block-encoding of $A^{-1}B$ we use the through the following lemma taken from [17]:

Lemma 1 (Product of block-encoded matrices)**.** *If U is an (α, a, δ)-block-encoding of an s-qubit operator A, and V is an (β, b, ϵ)-block-encoding of an s-qubit operator B then*[1] *$(I_b \otimes U)(I_a \otimes V)$ is an $(\alpha\beta, a+b, \alpha\epsilon+\beta\delta)$-block-encoding of AB.*

Proof.

$$\begin{aligned} &\|AB - \alpha\beta(\langle 0|^{\otimes a+b} \otimes I)(I_b \otimes U)(I_a \otimes V)(|0\rangle^{\otimes a+b} \otimes I)\| \\ &= \Big\|AB - \underbrace{\alpha(\langle 0|^{\otimes a} \otimes I)U(|0\rangle^{\otimes a} \otimes I)}_{\tilde{A}}\underbrace{\beta(\langle 0|^{\otimes b} \otimes I)V(|0\rangle^{\otimes b} \otimes I)}_{\tilde{B}}\Big\| \\ &= \|AB - \tilde{A}B + \tilde{A}B - \tilde{A}\tilde{B}\| \\ &= \|(A-\tilde{A})B + \tilde{A}(B-\tilde{B})\| \\ &\leq \|A-\tilde{A}\|\beta + \alpha\|B-\tilde{B}\| \\ &\leq \alpha\epsilon + \beta\delta. \end{aligned}$$

□

[1] The identity operators act on each others ancilla qubits, which is hard to express properly using simple tensor notation, but the reader should read this tensor product this way.

In order to detail the necessary block-encodings for B we start by looking at the term:

$$\begin{aligned} &C_{\eta,2} = \\ &-i\frac{\pi(r-q)\delta_{\tau_1}}{\hat{\delta}_\eta}\left(\hat{\eta}+\left(-\frac{1}{\eta_{\max}T(r-q)}\right)I_\eta\right) \\ &\quad\times \tilde{F}^\dagger_{c,1}\left(\hat{\eta}\right)\tilde{F}_{c,1}, \end{aligned} \tag{56}$$

We split it in two terms

$$D_{\eta,1} = \tilde{F}^\dagger_{c,1}\left(\hat{\eta}\right)\tilde{F}_{c,1} \tag{57}$$

and

$$D_{\eta,2} = -i\frac{\delta_{\tau_1}\pi(r-q)}{\hat{\delta}_\eta}\left(\hat{\eta}-\frac{1}{\eta_{\max}T(r-q)}I_\eta\right) \tag{58}$$

where δ_{τ_1} and $\hat{\delta}_\eta$ (See Section 3.4) are such that

$$\|D_{\eta,2}\| \leq 1. \tag{59}$$

We can now see how to implement unitary block-encodings of each $D_{\eta,1}$ and $D_{\eta,2}$ through linear combination of unitaries[12, 23]. To obtain the block-encoding of $C_{\eta,2}$, we can again make use of Theorem 1. Now, in order to implement $C_{\tau_1}+C_{\eta,2}$, we make it conceptually easier, although may not be resource optimal, by making use of the following abstraction for a linear combination of block-encodings:

Lemma 2 (Linear Combination of block-encoded matrices)**.** *If U is an (α,a,δ)-block-encoding of an s-qubit operator A, and V is an (β,b,ϵ)-block-encoding of an s-qubit operator B then we can block-encode $D=d_1A+d_2B$ by taking the following linear combination of unitaries*

$$d_1\alpha\left(I_b\otimes U\right)+d_2\beta\left(I_a\otimes V\right)$$

resulting in a $(|d_1|\alpha+|d_2|\beta, a+b+1, |d_1|\delta+|d_2|\epsilon)$-block-encoding of D.

Proof. By the results of [12, 23] we can block-encode

$$d_1\alpha\left(I_b\otimes U\right)+d_2\beta\left(I_a\otimes V\right)$$

through a unitary W with a subnormalization $|d_1|\alpha+|d_2|\beta$. The error on such block-encoding is the following:

$$\begin{aligned} &\|D-(|d_1|\alpha+|d_2|\beta)(\langle 0|^{\otimes a+b+1}\otimes I)W(|0\rangle^{\otimes a+b+1}\otimes I)\| \\ &=\left\|D-d_1\underbrace{\alpha(\langle 0|^{\otimes a}\otimes I)U(|0\rangle^{\otimes a}\otimes I)}_{\tilde{A}}+d_2\underbrace{\beta(\langle 0|^{\otimes b}\otimes I)V(|0\rangle^{\otimes b}\otimes I)}_{\tilde{B}}\right\| \\ &=\|d_1A-d_1\tilde{A}+d_2B-d_2\tilde{B}\| \\ \leq&|d_1|\delta+|d_2|\epsilon \end{aligned}$$

□

With this, we now have all the necessary ingredients to implement the block-encodings for the solution of the system. The table Table 2 summarizes the qubit requirements.

4 Extracting the amplitudes from the quantum memory

The final step of our algorithm is extracting amplitudes, $\psi(\tau_1,\eta)$, from the quantum memory which will provide us the final price of the option in question at the end of the time period, as well as in-between and for all values of the ratio $\eta\in[-\eta_{\max},\eta_{\max}]$. Here, we demonstrate our method of extraction in 6 sub-steps:

Operator	Qubits at first level block-encoding	Qubits at second level block-encoding
A_1^{-1} (inverse of term proportional to η^2, prefactor in second ratio derivative term)	1	0
C_{τ_1} (discretized time derivative)	2	0
$D_{\eta,1}$ (first part of second ratio derivative)	$\lceil \log_2(n_\eta + 1) \rceil$	0
$D_{\eta,2}$ (second part of ratio derivative term)	$\lceil \log_2(n_\eta + 1) \rceil$	0
$C_{\tau_1} + C_{\eta,2}$ (discretized time derivative + discretized ratio derivative term)	0	1
A^{-1} (inverse of the second ratio derivative)	0	1

Table 2: There is a requirement of l-qubits for both the oracle implementations of A_1 and A as well as the ancillar qubits necessary for the INV circuit, but we count them once because the oracle can be uncomputed after the INV circuit of the HHL algorithm.

- First, with a simplified one-variable problem, we demonstrate how one can sample an integral from quantum memory by means of binary decomposition/segmentation (Section 4.1).
- Second, we demonstrate how we can extract the functional form of the integral through interpolation by obtaining the integral for only a few values, provided the integral is a smooth function (Section 4.2).
- Third, we show that we can approximate the integrand by means of differentiating the interpolant. We also study the propagation of errors from the extraction of integral values (Section 4.3).
- Fourth, we demonstrate that the linear system we use to solve the interpolation problem of the second step corresponds to a stable matrix despite using sub-optimal interpolating nodes (Section 4.4).
- Fifth, we argue that one can take the square root of this approximation of the integrand without introducing discontinuities or blowing up the uncertanties by shifting the approximation just enough that is positive definite (Section 4.3).
- Finally, we show how this method extends to the problem of extracting the two-variable dependent solution $\psi(\eta, \tau_1)$ from quantum memory (Section 4.6).

One thing to get out of the way before talking about interpolating smooth integrals is the fact that the initial conditions contain a kink at $I = KT$ (or at $I = 0$ for average-strike options).

Since the differential Equation (1) is linear, parabolic differential equation, with a non-zero diffusion term, we expect the kink to smoothen-out for $t < T$. How smooth it becomes will depend on how big the diffusion term is and how far from the terminal condition. That is, in rough terms, after making the appropriate change of variables,

$$\left| \frac{\partial \psi}{\partial \eta} \right| = O\left(\frac{1}{\sigma \tau_1} \right). \tag{60}$$

In order to extract the amplitudes, we resort to Chebyshev interpolation. The are two issues to overcome: The nodes are distributed uniformly (not on Chebyshev collocation points), and the amplitudes are expected to be subnormalized by the size of the vector space.

For the first problem, we use mock Chebyshev interpolation which is almost as stable as Chebyshev interpolation, but whose collocation points are a subset of evenly spaced collocation points.

The second problem is solved by instead interpolating the integral

$$\Psi(\tau_1, \eta) = \int_{\Delta}^{\tau_1} \int_{-\eta_{\max}}^{\eta} \psi(\eta', \tau_1')^2 \mathrm{d}\eta' \mathrm{d}\tau_1' \tag{61}$$

on the mock-Chebyshev nodes for each variable.

4.1 Squared amplitude integral estimation with binary segmentation

Here, we explain how we can obtain the integral of probabilities in an efficient way through a type of binary decomposition and the use of the circular shift operator, S_x (See Appendix A for an alternative derivation).

For simplicity, suppose we have the following function: $\phi(x) = \langle\phi|x\rangle = \sqrt{p_x} > 0$, which depends on a single variable $x \in \{0, 1, 2, \ldots, 2^n - 1\}$. The state $|\phi\rangle$ encodes the function in a $N = 2^n$-size quantum memory the following way:

$$|\phi\rangle = \sum_{x=0}^{2^n-1} \sqrt{p_x}\,|x\rangle\,. \tag{62}$$

Suppose we need to sum the probabilities from x_i to x_f. We start by shifting to the left by x_i such that

$$S_{-x_i}\,|\phi\rangle = \sum_{x=0} \sqrt{p_{x_i+x}}\,|x\rangle \tag{63}$$

We then estimate the probability of measuring the first

$$m_0 = n - \lfloor \log_2 (x_f - x_i + 1) \rfloor \tag{64}$$

qubits on 0, which is

$$\begin{aligned} q_{x_1,x_i} &= \sum_{x=0}^{x_1-1} p_{x_i+x} \\ x_1 &= 2^{n-m_0} \end{aligned} \tag{65}$$

Following that, we check if

$$x_1 + x_i \leq x_f. \tag{66}$$

If so, we repeat the experiment, but with the larger shift

$$S_{-(x_i+x_1)}\,|\psi\rangle = \sum_{x=0} \sqrt{p_{x_i+x_1+x}}\,|x\rangle\,. \tag{67}$$

We then estimate the probability of measuring the first

$$m_1 = n - \lfloor \log_2 (x_f - (x_i + x_1) + 1) \rfloor \tag{68}$$

qubits on 0, which is

$$\begin{aligned} q_{x_2,x_1} &= \sum_{x=0}^{x_2-1} p_{x_i+x_1+x} \\ x_2 &= 2^{n-m_1} \end{aligned} \tag{69}$$

Following that, we check if

$$x_1 + x_2 + x_i \leq x_f. \tag{70}$$

We keep this going recursively, and finally we add the probabilities

$$\|q\|_1 \tag{71}$$

which gives us the sum of probabilities from x_i to x_f including endpoints, all this with $O(\log(x_f - x_i + 1))$ amplitude estimation experiments. This is possible because we can convert any integer into binary representation.

Provided we estimate the amplitudes through an amplitude estimation algorithm each with a target error ϵ', for each binary segment we will have:

$$\sqrt{\tilde{q}_{x_l,x_m}} = \sqrt{q_{x_l,x_m}} \pm \epsilon'. \tag{72}$$

After squaring and adding

$$\|\tilde{q}\|_1 = \|q\|_1 \pm \tilde{O}(\epsilon'). \tag{73}$$

Here, $\tilde{O}(\cdot)$ means we ignore any polylog$(\cdot)$ factors.

We also note that,

$$\|q\|_1 \approx 2^n \int_{y_i}^{y_f} \phi^2(y)\mathrm{d}y \tag{74}$$

with the change of variables $y = x\delta_y$, $\delta_y = 1/2^n$, and where

$$\phi(y) = \langle y|\phi\rangle, \tag{75}$$

with the relabeled states. Thus,

$$\frac{1}{2^n}\|\tilde{q}\|_1 = \int_{y_i}^{y_f} \phi^2(y)\mathrm{d}y \pm \tilde{O}\left(\frac{\epsilon'}{2^n}\right). \tag{76}$$

As we mentioned before, the interpolant we want to obtain approximates this integral in between interpolant nodes. In the next subsection we will illustrate how we accomplish this.

4.2 Estimating the integral Cebyshev expansion with a few evaluations

Defining

$$\Phi = \int_0^y \phi(y')^2\mathrm{d}y' \tag{77}$$

we can now use Chebyshev interpolation to obtain the expansion coefficients of the interpolant through the following linear system:

$$\mathbf{V}a = f, \tag{78}$$

where $(f)_k = \Phi(y_k)$, and y_k are the interpolation nodes. We make a further change of variables through a linear transformation such that we map $y \to s \in [-1, 1]$. We have that

$$\mathbf{V} := \begin{pmatrix} u_0(s_1) & u_1(s_1) & \dots & u_{M-1}(s_1) \\ u_0(s_2) & u_1(s_2) & \dots & u_{M-1}(s_2) \\ \vdots & \vdots & \ddots & \vdots \\ u_0(s_M) & u_1(s_M) & \dots & u_{M-1}(s_M) \end{pmatrix}. \tag{79}$$

With this, one can obtain the coefficients, a_j, with $a = \mathbf{V}^{-1}f$. Like in [37, 35], the choice of interpolating nodes s_k and the interpolating set of polynomials are the Chebyshev nodes and polynomials. In that case, u_j is defined by

$$u_j(s) := \begin{cases} \sqrt{\frac{1}{M}}T_0(s), & j = 0 \\ \sqrt{\frac{2}{M}}T_j(s), & j = 1, 2, \dots, M-1 \end{cases} \tag{80}$$

where T_j is the standard jth Chebyshev polynomial.

$$T_j(s) := \cos(j \cos^{-1} s) \tag{81}$$

The node collocation is described by

$$s_k = \cos\left(\frac{2k-1}{2M}\pi\right), k \in \{1, 2, \dots, M\}. \tag{82}$$

These polynomials fulfill the discrete orthonormality condition [22] with respect to the collocation nodes, that is,

$$\sum_{k=1}^{M} u_i(s_k)u_j(s_k) = \delta_{ij} \tag{83}$$

for all $0 \leq i, j < M$. With this we know that the condition number for $\mathbf{V}$ is optimal, or $\kappa(\mathbf{V}) = \sigma_{\max}(\mathbf{V})/\sigma_{\min}(\mathbf{V}) = 1$.

4.3 Extracting the integrand ϕ^2

Here, we explain how we obtain an approximation of the integrand ϕ^2 out of the interpolant for $\Phi(y)$ (See Equation (77)). We also show how the errors form the estimated $\Phi(y)$ propagate towards this approximant of ϕ^2.

First, we note that are actually going to obtain the coefficients from

$$\mathbf{V}_{\text{pert}} a' = f', \tag{84}$$

where f' is the integral samples with errors introduced from the finite-resources amplitude estimation. $\mathbf{V}_{\text{pert}}$ is the a slightly perturbed Chebyshev-Vandermonde matrix:

$$\mathbf{V}_{\text{pert}} := \begin{pmatrix} u_0(s'_1) & u_1(s'_1) & \dots & u_{M-1}(s'_1) \\ u_0(s'_2) & u_1(s'_2) & \dots & u_{M-1}(s'_2) \\ \vdots & \vdots & \ddots & \vdots \\ u_0(s'_M) & u_1(s'_M) & \dots & u_{M-1}(s'_M) \end{pmatrix}. \tag{85}$$

where the perturbation comes from obtaining the amplitudes at slightly-off collocation nodes, s'_k. This comes from using a finite size grid, even though exponential in n. First, we look at the error on the vector of amplitudes coming from amplitude estimation. After re-scaling by the factor 2^n we note that for each element:

$$|(f - f')_j| = \tilde{O}\left(\frac{\epsilon'}{2^n}\right), \tag{86}$$

where ϵ' is the error with which we estimate each $\|q\|_1$ for the different collocation nodes. Now, we look at how the error is propagated to the expansion coefficients:

$$\|a - a'\|_1 = \|V^{-1}_{\text{perturbed}}(f - f')\|_1 \tag{87}$$

Using Schwarz inequality

$$\begin{aligned} \|a - a'\|_1 &= \|V^{-1}_{\text{perturbed}}(f - f')\|_1 \\ &= \|V^{-1}_{\text{perturbed}}\|_2 \|(f - f')\|_2 \\ &= O\left(\frac{\sqrt{M}\epsilon'}{2^n}\right) \end{aligned} \tag{88}$$

Here, we have assumed $\|V^{-1}_{\text{perturbed}}\|_2 = O(1)$; more on that, later. Ignoring the algorithmic error from using a finite size (M) interpolation, we know that the propagated uncertainty is upper bounded by:

$$\|(a - a')D_s u(s)\|_1. \tag{89}$$

Here,

$$(D_s u(s))_n = \frac{\mathrm{d}(u(s))_n}{\mathrm{d}s} = \begin{cases} \sqrt{\frac{\pi}{M}} \frac{\mathrm{d}T_n(s)}{\mathrm{d}s} & n = 0 \\ \sqrt{\frac{\pi}{M}} \frac{\mathrm{d}T_n(s)}{\mathrm{d}s} & n \in \{1, \dots, M-1\} \end{cases}. \tag{90}$$

Moreover, we note that the derivative of $T_n(x)$ is

$$\frac{\mathrm{d}T_n(s)}{\mathrm{d}s} = nU_n(s) \tag{91}$$

and we also note that

$$\max_s |U_n(s)| \leq n+1, \tag{92}$$

which in turn means that

$$\max_s \left| \frac{\mathrm{d}T_n(s)}{\mathrm{d}s} \right| \leq n^2 + n. \tag{93}$$

Thus, after differentiating the interpolant, we expect to have an error

$$\tilde{O}\left(\frac{M^2\epsilon'}{2^n}\right). \tag{94}$$

4.4 Perturbation on linear system matrix, V

Regarding the perturbed matrix $\mathbf{V}_{\text{pert}}$, we assumed that the norm of its inverse was $\|\mathbf{V}_{\text{pert}}^{-1}\|_2 = O(1)$. The orginal Chebyshev-Vandermonde, $\mathbf{V}_{\text{cheb}}$, matrix has all eigenvalues $\lambda_j = 1$, hence the stability. However, we are limited to collocating the sampling nodes at equi-spaced distance. To get an idea on why we can assume $\|\mathbf{V}_{\text{pert}}^{-1}\|_2 = O(1)$, we first consider the upper bounds on eigen-value perturbation:

$$|\lambda_j - \lambda| \leq \|\mathbf{V} - \mathbf{V}_{\text{pert}}\|_2. \tag{95}$$

Now, in order to estimate the size of the spectral perturbation on the matrix, we look at the perturbation on each of its elements:

$$\begin{aligned} \|\mathbf{V} - \mathbf{V}_{\text{pert}}\|_{\max} &= \max_{k,n \leq M-1} |u_n(s_k) - u_n(s'_k)| \\ &\leq |s_k - s'_k| \max_{s\in[-1,1]} |D_s u_n(s)|. \end{aligned} \tag{96}$$

Here, M stands for either M_{τ_1} or M_η, the number of interpolation points for each variable. We know that the number of accessible node collocation points are N_η and $N'_{\tau_1} = O(N_{\tau_1})$, the size of the memory spanned by $t \in [\Delta, T]$. Thus, the error on the collocation points is:

$$|s_k - s'_k| = O(1/N), \tag{97}$$

where N stands for either N_η or N'_{τ_1}. Moreover, we have already established that

$$\max_s \left| \frac{\mathrm{d}T_n(s)}{\mathrm{d}s} \right| = O(n^2). \tag{98}$$

This means the maximum norm on the perturbation is:

$$\|\mathbf{V} - \mathbf{V}_{\text{pert}}\|_{\max} = O\left(\frac{M^{3/2}}{N}\right). \tag{99}$$

Thus, the corresponding spectral norm is:

$$\begin{aligned} \|\mathbf{V} - \mathbf{V}_{\text{pert}}\|_2 &\leq M\|\mathbf{V} - \mathbf{V}_{\text{pert}}\|_{\max} \\ &= O\left(\frac{M^{5/2}}{N}\right). \end{aligned} \tag{100}$$

In each case for both variables, we expect this to be very small since both N_η and N'_{τ_1} grow exponentially with the number of qubits. For that reason, it is safe to assume, for each variable, that the perturbation is small and thus $\|\mathbf{V}_{\text{pert}}\|_2 = O(1)$.

4.5 Taking the square root of the approximant of ϕ^2

Here, we briefly explain how we are able to take the square root of the approximant of ϕ^2 without introducing any discontinuities and large uncertainties.

We need to take the square root of this estimated quantity and estimate the propagated uncertainty. The true value is positive definite, thus, if at any point the approximant is less than or equal to zero, we need only shift it upwards with something of the same order as the error until it is positive. After this shift and the square root operation, we expect that the error is

$$\tilde{O}\left(\frac{M^2\epsilon'}{2^n \min_s \phi(s)}\right). \tag{101}$$

4.6 Extending this to the case of $\psi(\eta, \tau_1)$

We have explained the algorithm for the extraction of an amplitude $\phi(x)$ which is a smooth function over one variable. Here, we go back to the full problem at hand, that of Asian option pricing, for which we expect a two-variable interpolation (t and η). Thus, after the two-variable interpolation and the corresponding two differentiation operations, the expected error on the new interpolant is:

$$\tilde{O}\left(\frac{M_\eta^2 M_{\tau_1}^2 \epsilon'}{2^{n_{\tau_1}}}\right). \tag{102}$$

We note that there is only a factor $1/2^{n_{\tau_1}}$ and not $1/2^{n_\eta}$ also because of the normalization factor $1/\sqrt{\mathcal{N}_b}$ on $|\tilde{\psi}\rangle$, where $\mathcal{N}_b = O\left(\eta_{\max}^3 2^{n_\eta}\right)$. We then shift if necessary to make this quantity positive definite, introducing only an error of similar magnitude.

Finally, after taking the square root of the doubly-differentiated and positive-definite, the propagated uncertainty is:

$$\tilde{O}\left(\frac{M_\eta^2 M_{\tau_1}^2}{\min_{\eta,t\in I_{\tau_1}} \psi} \frac{\epsilon'}{2^{n_{\tau_1}}}\right), \tag{103}$$

where ψ is the true function and I_{τ_1} is the sub-interval, $[\Delta, T]$, in the domain of t for which we think ψ is sufficiently smooth.

With this, if we want that the final target error on the interpolant of ψ is ϵ, we must estimate each sample of Ψ with uncertainty

$$\tilde{O}\left(\frac{\min_{\eta,t\in I_{\tau_1}} \psi}{M_\eta^2 M_{\tau_1}^2} \epsilon 2^{n_{\tau_1}}\right). \tag{104}$$

Thus, the total cost of sampling all the $M_\eta M_{\tau_1}$ samples of Ψ with said uncertainty is

$$\tilde{O}\left(\frac{1}{2^{n_{\tau_1}}\epsilon} \frac{M_\eta^3 M_{\tau_1}^3}{\min_{\eta,t\in I_{\tau_1}} \psi}\right). \tag{105}$$

We note that the factors M_η and M_{τ_1} are $\tilde{O}(\log 1/\epsilon)$ since we assume smoothness and no other big scale involved and ignore them from now on as well. We also note that through the $\tilde{O}(\cdot)$-notation we have ignored the factors proportional to poly (σ), poly $(\eta_{\max})$, and poly (T).

From here, we point out that we can decrease ϵ exponentially fast while keeping costs $\tilde{O}\left(\text{polylog}\left(1/\epsilon\right)\right)$ provided that we increase

$$n_{\tau_1} = O\left(\log\left(\frac{1}{\epsilon \min_{\eta,t\in I_{\tau_1}} \psi}\right)\right). \tag{106}$$

5 Algorithm Summary

To recapitulate, in rough terms, this is the summary of our algorithm:

- We apply the unitary that block-encode the inverse of A_1 fastly using HHL algorithm
- We apply the unitary that block-encode the inverse of A fastly using HHL algorithm
- We find the QSVT phases to implement the inverse of W, the preconditioned matrix.
- We apply the unitary block-encoding of the inverse of W.
- We then extract the integral of the square amplitudes $\Psi(\tau_1, \eta) = \int\int \psi^2 \mathrm{d}\eta\, \mathrm{d}\tau_1$ from the quantum memory at the Chebyshev nodes for τ_1 and η.
- We perform the interpolation of Ψ with these extracted integrals.
- Differentiate the interpolant of Ψ with respect to η and τ_1 to obtain an approximation of ψ^2.
- Shift the approximation of ψ^2 so that it is strictly positive.
- Finally, take the square root of that approximant to estimate $\psi(\tau_1, \eta)$.

Algorithm 2 $U = U_{W^{-1}} U_{A^{-1}} U_{A_1^{-1}} U_b$. Here, $\rho = |0^{\otimes(n_{\tau_1}+n_\eta+\text{ancilla})}\rangle\langle 0^{\otimes(n_{\tau_1}+n_\eta+\text{ancilla})}|$ and G selects the block-encodings in U.

Require: $\|A_1^{-1}\| = O(1)$, $\|A_2^{-1}\| = O(1)$
Require: $\|C_{\tau_1} \otimes I_\eta + I_{\tau_1} \otimes C_{\eta,2}\| = O(1)$
Require: $|\hat{b}\rangle = |b\rangle / \| |b\rangle \| = U_b |0\rangle$
Require: $\| \left(I_{\tau_1} \otimes A_2 + C_{\tau_1} \otimes A_1^{-1} + I_{\tau_1} \otimes A_1^{-1} C_{\eta,2}\right)^{-1} \| = O(1)$

```
Estimate the phases required for QSVT to implement W^{-1} and sub-normalization α_{W^{-1}} of its implementation
for k ∈ {1, ..., M_η} do
    Binary-decompose s_k^{(η)} according to Section 4.1 into segments delimited by {m_0^{(η)}, ... m_{-1}^{(η)}}
    for l ∈ {1, ..., M_{τ_1}} do
        Binary-decompose s_k^{(τ_1)} according to Section 4.1 into segments delimited by {m_0^{(τ_1)}, ... m_{-1}^{(τ_1)}}
        for i do
            for j do
                Estimate Tr( G |0^{⊗m_j^{(η)}}⟩⟨0^{⊗m_j^{(η)}}|_η |0^{⊗m_i^{(τ_1)}}⟩⟨0^{⊗m_i^{(τ_1)}}|_{τ_1} U ρ U^† )
            end for
        end for
    end for
end for
Scale the estimated traces by the normalization constants (N_b, α_{A_1^{-1}}, α_{A_2^{-1}}, α_{W^{-1}}).
Find interpolation coefficients for the double integral.
Differentiate interpolant with respect to both η and τ_1.
Shift derivative if necessary for positive-definiteness.
Take the square root of derivative of interpolant after evaluating at specific η and τ_1.
```

6 Discussion & Conclusions

We demonstrate that a quantum advantage is possible for Asian options pricing which has a scaling of $\tilde{O}(\text{polylog}\,(1/\epsilon))$ whereas classical methods, for example, scale like $\tilde{O}(1/\epsilon^3)$ using Monte Carlo methods and $\tilde{O}(1/\epsilon^2)$ if using finite-difference with efficient tri-diagonal discretizations for η. The best classical

discretization methods achieve $\tilde{O}(1/\epsilon)$, thus, even comparing with the best classical algorithm we achieve an exponential improvement. Moreover, we achieve an exponential improvement over other existing quantum algorithms with our probability integral estimation and interpolation. This was due to the bottle-neck other quantum algorithms suffered when extracting the information from the prepared state vector. Here, we used the smoothness of the solution over the intervals of interest to achieve an exponentially converging interpolation and our probability integral estimation to achieve an exponential precision over each integral estimation.

From the expansion of ψ we can obtain the greeks Δ and Θ without extra quantum cost and negligible amount of classical operations with an exponentially small precision as well. If we assume that the value of the option is smooth with respect to the parameter σ, we can also estimate $\mathcal{V}$ with a few, $O(\log(1/\epsilon))$, repetitions to obtain a Chebyshev expansion of the derivative with respect to σ.

In the future, we hope to come up with more systematic ways of adapting these type of methods for other differential equations. Moreover, it would be beneficial to get a handle of singularities using these methods as well and also adapt the work here for non-linear differential equations. Also, an existing bottle-neck in algorithms using fast-inversion at the moment is the method used to compute the reciprocals of the eigenvalues stored (See Section 3.3), for example a Newton iterative method in [8] which has a $O\left(\log^4 1/\epsilon\right)$ cost and a $O\left(\log^3 1/\epsilon\right)$ number of ancilla qubits required [50]. Finally, we hope to extend these methods to Asian option pricing in the context of the Heston model where one more variable is introduced (σ becomes a stochastic variable).

References

[1] Javier Alcazar, Andrea Cadarso, Amara Katabarwa, Marta Mauri, Borja Peropadre, Guoming Wang, and Yudong Cao. Quantum algorithm for credit valuation adjustments. *New Journal of Physics*, 24(2):023036, February 2022.

[2] Gadi Aleksandrowicz, Thomas Alexander, Panagiotis Barkoutsos, Luciano Bello, Yael Ben-Haim, David Bucher, F Jose Cabrera-Hernández, Jorge Carballo-Franquis, Adrian Chen, Chun-Fu Chen, et al. Qiskit: An open-source framework for quantum computing. *Accessed on: Mar*, 16:61, 2019.

[3] Dong An, Noah Linden, Jin-Peng Liu, Ashley Montanaro, Changpeng Shao, and Jiasu Wang. Quantum-accelerated multilevel monte carlo methods for stochastic differential equations in mathematical finance. *Quantum*, 5:481, June 2021.

[4] Babuška, I. and Suri, Manil. The h - p $h-p$ version of the finite element method with quasiuniform meshes. *ESAIM: M2AN*, 21(2):199–238, 1987.

[5] Adam Bouland, Aditi Dandapani, and Anupam Prakash. A quantum spectral method for simulating stochastic processes, with applications to monte carlo, 2023.

[6] Gilles Brassard, Peter Hoyer, Michele Mosca, and Alain Tapp. Quantum Amplitude Amplification and Estimation. *arXiv e-prints*, pages quant–ph/0005055, May 2000.

[7] V. Buzek, R. Derka, and S. Massar. Optimal quantum clocks. *Phys. Rev. Lett.*, 82:2207–2210, 1999.

[8] Yudong Cao, Anargyros Papageorgiou, Iasonas Petras, Joseph Traub, and Sabre Kais. Quantum algorithm and circuit design solving the poisson equation. *New Journal of Physics*, 15(1):013021, jan 2013.

[9] Zhongdi Cen, Aimin Xu, and Anbo Le. A hybrid finite difference scheme for pricing asian options. *Applied Mathematics and Computation*, 252:229–239, 2015.

[10] Shouvanik Chakrabarti, Rajiv Krishnakumar, Guglielmo Mazzola, Nikitas Stamatopoulos, Stefan Woerner, and William J. Zeng. A threshold for quantum advantage in derivative pricing. *Quantum*, 5:463, June 2021.

[11] Andrew M. Childs, Jin-Peng Liu, and Aaron Ostrander. High-precision quantum algorithms for partial differential equations. *Quantum*, 5:574, November 2021.

[12] Andrew M Childs and Nathan Wiebe. Hamiltonian simulation using linear combinations of unitary operations. *Quantum Information & Computation*, 12(11-12):901–924, 2012.

[13] R. Cleve, A. Ekert, C. Macchiavello, and M. Mosca. Quantum algorithms revisited. *Proceedings of the Royal Society of London Series A*, 454(1969):339, January 1998.

[14] J.P.A. Devreese, D. Lemmens, and J. Tempere. Path integral approach to asian options in the black–scholes model. *Physica A: Statistical Mechanics and its Applications*, 389(4):780–788, February 2010.

[15] JN Dewynne and WT Shaw. Differential equations and asymptotic solutions for arithmetic asian options:'black–scholes formulae'for asian rate calls. *European Journal of Applied Mathematics*, 19(4):353–391, 2008.

[16] Filipe Fontanela, Antoine Jacquier, and Mugad Oumgari. A quantum algorithm for linear pdes arising in finance, 2021.

[17] András Gilyén, Yuan Su, Guang Hao Low, and Nathan Wiebe. Quantum singular value transformation and beyond: exponential improvements for quantum matrix arithmetics. In *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing*, pages 193–204, 2019.

[18] Aram W Harrow, Avinatan Hassidim, and Seth Lloyd. Quantum algorithm for linear systems of equations. *Physical review letters*, 103(15):150502, 2009.

[19] Dylan Herman, Cody Googin, Xiaoyuan Liu, Yue Sun, Alexey Galda, Ilya Safro, Marco Pistoia, and Yuri Alexeev. Quantum computing for finance. *Nature Reviews Physics*, 5(8):450–465, July 2023.

[20] J. Lars Kirkby and Duy Nguyen. Efficient Asian option pricing under regime switching jump diffusions and stochastic volatility models. *Annals of Finance*, 16(3):307–351, September 2020.

[21] Natalie Klco and Martin J Savage. Digitization of scalar fields for quantum computing. *Physical Review A*, 99(5):052335, 2019.

[22] Mykhailo Kuian, Lothar Reichel, and Sergij Shiyanovskii. Optimally conditioned vandermonde-like matrices. *SIAM Journal on Matrix Analysis and Applications*, 40(4):1399–1424, 2019.

[23] Guang Hao Low and Isaac L Chuang. Hamiltonian simulation by qubitization. *Quantum*, 3:163, 2019.

[24] A. Luis and J. Peřina. Optimum phase-shift estimation and the quantum description of the phase difference. *Phys. Rev. A*, 54:4564–4570, Nov 1996.

[25] Manisha and S. Chandra Sekhara Rao. A computational technique for asian option pricing model. In João M. F. Rodrigues, Pedro J. S. Cardoso, Jânio Monteiro, Roberto Lam, Valeria V. Krzhizhanovskaya, Michael H. Lees, Jack J. Dongarra, and Peter M.A. Sloot, editors, *Computational Science – ICCS 2019*, pages 326–339, Cham, 2019. Springer International Publishing.

[26] Koichi Miyamoto. Bermudan option pricing by quantum amplitude estimation and chebyshev interpolation. *EPJ Quantum Technology*, 9(1):1–27, 2022.

[27] Ashley Montanaro. Quantum speedup of monte carlo methods. *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 471(2181):20150301, 2015.

[28] Ashley Montanaro and Sam Pallister. Quantum algorithms and the finite element method. *Physical Review A*, 93(3):032324, 2016.

[29] Román Orús, Samuel Mugel, and Enrique Lizaso. Quantum computing for finance: Overview and prospects. *Reviews in Physics*, 4:100028, November 2019.

[30] Anupam Prakash, Yue Sun, Shouvanik Chakrabarti, Charlie Che, Aditi Dandapani, Dylan Herman, Niraj Kumar, Shree Hari Sureshbabu, Ben Wood, Iordanis Kerenidis, and Marco Pistoia. Quantum option pricing via the karhunen-loève expansion, 2024.

[31] Santosh Kumar Radha. Quantum option pricing using wick rotated imaginary time evolution, 2021.

[32] Patrick Rebentrost, Brajesh Gupt, and Thomas R. Bromley. Quantum computational finance: Monte carlo pricing of financial derivatives. *Physical Review A*, 98(2), August 2018.

[33] Patrick Rebentrost, Brajesh Gupt, and Thomas R Bromley. Quantum computational finance: Monte carlo pricing of financial derivatives. *Physical Review A*, 98(2):022321, 2018.

[34] Patrick Rebentrost, Alessandro Luongo, Samuel Bosch, and Seth Lloyd. Quantum computational finance: martingale asset pricing for incomplete markets, 2022.

[35] Gumaro Rendon. All you need is trotter, 2024.

[36] Gumaro Rendon, Taku Izubuchi, and Yuta Kikuchi. Effects of cosine tapering window on quantum phase estimation. *Phys. Rev. D*, 106:034503, Aug 2022.

[37] Gumaro Rendon, Jacob Watkins, and Nathan Wiebe. Improved Accuracy for Trotter Simulations Using Chebyshev Interpolation. *Quantum*, 8:1266, February 2024.

[38] L. Rogers and Z. Shi. The value of an asian option. *Journal of Applied Probability*, 32:1077–1088, 12 1995.

[39] M. Sander. *Bondesson's Representation of the Variance Gamma Model and Monte Carlo Option Pricing.* Master's theses in mathematical sciences. Lund University, 2009.

[40] Jie Shen and Haijun Yu. Efficient spectral sparse grid methods and applications to high-dimensional elliptic problems. *SIAM Journal on Scientific Computing*, 32(6):3228–3250, 2010.

[41] Jie Shen and Haijun Yu. Efficient spectral sparse grid methods and applications to high-dimensional elliptic equations ii. unbounded domains. *SIAM Journal on Scientific Computing*, 34(2):A1141–A1164, 2012.

[42] Rolando D Somma. Quantum simulations of one dimensional quantum systems. *Quantum Information & Computation*, 16(13-14):1125–1168, 2016.

[43] Nikitas Stamatopoulos, Daniel J. Egger, Yue Sun, Christa Zoufal, Raban Iten, Ning Shen, and Stefan Woerner. Option pricing using quantum computers. *Quantum*, 4:291, July 2020.

[44] Nikitas Stamatopoulos, Guglielmo Mazzola, Stefan Woerner, and William J. Zeng. Towards quantum advantage in financial market risk using quantum gradient algorithms. *Quantum*, 6:770, July 2022.

[45] Nikitas Stamatopoulos and William J. Zeng. Derivative pricing using quantum signal processing. *Quantum*, 8:1322, April 2024.

[46] Yu Tong, Dong An, Nathan Wiebe, and Lin Lin. Fast inversion, preconditioned quantum linear system solvers, fast green's-function computation, and fast evaluation of matrix functions. *Physical Review A*, 104(3):032422, 2021.

[47] Guoming Wang and Angus Kan. Option pricing under stochastic volatility on a quantum computer, 2024.

[48] Erik Wiklund. Asian option pricing and volatility. 2012.

[49] Mark-Oliver Wolf, Roman Horsky, and Jonas Koppe. A quantum algorithm for pricing asian options on valuation trees. *Risks*, 10(12), 2022.

[50] Shilu Yan, Tong Dou, Runqiu Shu, and Wei Cui. Module for arbitrary controlled rotation in gate-based quantum algorithms. *arXiv preprint arXiv:2107.08168*, 2021.

A Amplitude Extraction Simplified

Lemma 3 (Probability Integral Estimation)**.** *Given a statevector with positive-definite amplitudes*

$$|\phi\rangle = \sum_{x}^{2^n-1} \phi(x)\,|x\rangle \tag{107}$$

we can estimate the integral

$$\Phi(\hat{y}) = \int_0^{\hat{y}} \phi^2(\hat{x})\mathrm{d}\hat{x} \tag{108}$$

where $\hat{x} = x/2^n$*,* $\hat{y} = y/2^n < 1$ *, and* $y \in \mathbb{N}$*, within an error* ϵ *with a gate complexity of*

$$\tilde{O}\left(\frac{1}{2^n\epsilon}\right)$$

Proof. We can estimate the integral with the following left-Riemann sum:

$$\Phi(\hat{y}) = \frac{1}{2^n}\sum_{x=0}^{y-1} \phi^2(x) + O\left(\frac{1}{2^n}\right) \tag{109}$$

We consider $\hat{y} = y/2^n$, for which we can always find the following binary decomposition:

$$y = y_0 + 2y_1 + \cdots + 2^{m-1}y_{m-1}. \tag{110}$$

where $y_j \in \{0, 1\}$ and we truncate this expansion so that $y_{m-1} = 1$.

We set $w = 0$, then, for $j \in \{m-1, m-2, \ldots, 0\}$, we do:

- If $y_j = 1$:
  - Estimate $\sum_{x=w}^{w+2^j-1} \phi^2(x)$ through the probability of measuring all zeros from the j_{th} bit to the right after circularly shifting the register "down" by w.
  - Add 2^j to w.

Thus, we only need to do at most $m \leq n$ quantum amplitude estimations of target error ϵ to obtain an estimate of

$$\sum_{x=0}^{y-1} \phi^2(x) \tag{111}$$

with error at most:

$$O(n\epsilon). \tag{112}$$

Finally, to obtain an estimate of $\Phi(\hat{y})$ we divide by this estimate by 2^n and the error bound gets divided by 2^n as well:

$$\tilde{O}\left(\frac{\epsilon}{2^n}\right), \tag{113}$$

where $\tilde{O}(\cdot)$ means we have neglected factors of n. □

With this result, and through the circular shift operator S_w we can always approximate the definite integral with integration limits $[\hat{a}, \hat{b}]$.

Corollary 4. *Given a statevector with positive-definite amplitudes*

$$|\phi\rangle = \sum_{x}^{2^n-1} \phi(x)\,|x\rangle \tag{114}$$

we can estimate the integral

$$\Phi(\hat{y}) = \int_{\hat{a}}^{\hat{b}} \phi^2(\hat{x})\mathrm{d}\hat{x} \tag{115}$$

where $\hat{x} = x/2^n$, $\hat{a} = a/2^n < 1$, $\hat{b} = b/2^n < 1$, *and* $a, b \in \mathbb{N}$, *within an error* ϵ *with a gate complexity of*

$$\tilde{O}\left(\frac{1}{2^n\epsilon}\right)$$

B η Derivative discretization errors

Recalling the final linear equation we are to solve from Eq. (45):

$$(C_{\tau_1} \otimes I_\eta + I_{\tau_1} \otimes C_{\eta,1} + I_{\tau_1} \otimes C_{\eta,2})\,|\tilde{\psi}\rangle = |\hat{b}\rangle\,, \tag{116}$$

where

$$\begin{aligned} |\hat{b}\rangle &= \frac{|b\rangle}{\sqrt{\mathcal{N}_b}}, \\ |\tilde{\psi}\rangle &= \frac{|\psi\rangle}{\sqrt{\mathcal{N}_b}}. \end{aligned} \tag{117}$$

In this equation, provided that $\psi_0(\eta)$ is continuous and the kinks are off the nodes, the error form the η-second derivative approximation is at most $O(\delta_{\tau_1}\|\tilde{\psi}\|_\infty/\hat{\delta}_\eta)$ (for the first derivative part it is $O(\delta_{\tau_1}\|\tilde{\psi}\|_\infty)$). We have that $\sqrt{\mathcal{N}_b} = O\left(1/\sqrt{\hat{\delta}_\eta}\right)$, thus the error on the equation due to the discretized η operators is

$$\|\epsilon_\eta\|_\infty = O\left(\delta_{\tau_1}\|\psi\|_\infty/\hat{\delta}_\eta^{3/2}\right). \tag{118}$$

Since

$$\|\,(C_{\tau_1} \otimes I_\eta + I_{\tau_1} \otimes C_{\eta,1} + I_{\tau_1} \otimes C_{\eta,2})^{-1}\,\| = \tilde{O}(1), \tag{119}$$

the error on the solution due to the discretized η operators is also

$$\tilde{O}\left(\delta_{\tau_1}\|\psi\|_\infty/\hat{\delta}_\eta^{3/2}\right), \tag{120}$$

which is vanishingly small considering we use the condition

$$\delta_{\tau_1} = \Theta\left(\frac{\hat{\delta}_\eta^2}{\sigma^2}\log 1/\epsilon\right) \tag{121}$$

in Eq. (51). Thus, the error is

$$\tilde{O}\left(\hat{\delta}_\eta^{1/2}\|\psi\|_\infty\right). \tag{122}$$

Note that it is important that the initial condition $\psi_0(\eta)$ is continuous because the error expected on the linear equation due to the second derivative discretization of η near a discontinuity is $O(\delta_{\tau_1}\|\tilde{\psi}\|_\infty/\hat{\delta}_\eta^2)$, which does not vanish rapidly enough with our choice of scale relations in Eq. (51).

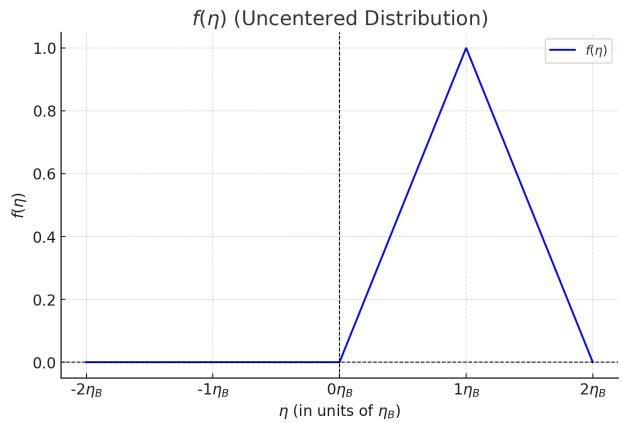

Figure 5: Continuous Initial Condition

B.1 Discontinuous State Vector, $\psi_0(\eta)$

Suppose that the initial state is $\psi(\eta, \tau_1 = 0) = \max(0, \eta)$ for the average-rate call case Section 2.1; because of the periodization implied for the η derivative discretizations, a discontinuity is also implied. For us to ensure a continuous function, we can truncate the linear function at $\eta = \eta_{\max}/2$ and complete it with a descending linear function with same slope like in Fig. 5. That is, we instead use $\psi_0(\eta) = \frac{\eta_{\max}}{2}\Lambda\left(\frac{2\eta}{\eta_{\max}} - \frac{1}{2}\right)$, where the triangular function is defined by:

$$\Lambda(x) = \begin{cases} 0 & |x| \geq 1 \\ 1 - |x| & |x| < 1. \end{cases} \tag{123}$$

This way, if $N_\eta/4 \in \mathbb{Z}_+$ we guarantee that the original kink and the ones introduced by this change are off the collocation nodes for η.

As the state evolves in time, the kinks move around, but also get smoothened out and so we do not have to worry about the convergence of errors due to η discretization. That is, the higher frequencies are smoothened as

$$e^{-\Omega(\sigma^2 k^2 \delta_{\tau_1})} \tag{124}$$

where k is the wave number of the Fourier component. So as long as

$$\sigma^2 \frac{N_\eta^2}{N_{\tau_1}} = \Omega\left(\log 1/\epsilon\right), \tag{125}$$

we do not have to worry about any kinks or discontinuities of the solution, $\psi(\eta, \tau_1)$, after the initial time.